CIS 3353: Computer Systems Security

Group Project Kick-off Guide

University of the Incarnate Word
Spring 2026

Based on: CompTIA Security+ Guide to Network Security Fundamentals
by Mark Ciampa | 8th Edition | SY0-701 | Copyright 2025

Version 3.0 — Updated for 8th Edition Module Structure

Welcome

Welcome to the CIS 3353 Group Project. This guide walks you through forming your team, selecting a project scenario, and setting up Agile project management on GitHub. The project requires you to apply concepts from the CompTIA Security+ curriculum to a practical, hands-on security implementation.

This version of the guide has been updated to reflect the 8th Edition module structure and provides detailed explanations of Agile concepts (Milestones, Sprints, User Stories, Issues, and Story Points) for students who have not used GitHub or Agile before.

> **Deadlines**
>
> All project deadlines are published on Canvas. The dates mentioned in this guide are for illustration only. Always refer to Canvas for the official due dates.

Section 1: Team Formation & Project Foundation

Part 1: Assemble Your Security Team

Team Size: Teams must consist of 2–3 members.

Skill Assessment: Discuss your team's interests. This is not about prior expertise but about willingness to learn:

- Who is interested in network design and architecture?
- Who wants to focus on hands-on system administration (Windows/Linux)?
- Who is drawn to cryptography and secure communications?
- Who is organized and would make a good project manager?
- Who enjoys writing and can lead the documentation efforts?

Finalize Your Team: Once your team is formed, designate one person to post all team member names to the "Team Formation" discussion board on Canvas.

Initial Role Delegation

Assign initial leads to keep the team focused. These roles define who coordinates each area, but they do not define who does the work.

> **Important: Everyone Does Technical Work**
>
> Every team member must contribute to the technical implementation (building, configuring, and testing systems). No one may limit their contribution to documentation or project management alone. Roles are about coordination, not division of labor.

- **Project Lead / Manager:** Coordinates the GitHub Project board, tracks sprint progress, and ensures deadlines are met. Also contributes to technical implementation.
- **System Architect / Engineer:** Leads the design and hands-on implementation of VMs, networks, and services. Also contributes to documentation.
- **Security Analyst / Documentation Lead:** Leads the security configuration testing and coordinates documentation in the GitHub Wiki. Also contributes to system setup and configuration.

Section 2: Choosing Your Security Project

Part 2: Brainstorming Your Security Project

Your project must be a practical implementation that combines concepts from at least three different modules covered in the course. The goal is to design, build, and document a secure system or environment that addresses a specific security challenge.

Step 1: Review the Course Modules (8th Edition)

The table below shows all 15 modules from the textbook. Modules 1–10 are covered in class. Modules 11–15 are not covered in lectures but you may explore them for your project.

Module	Title	SY0-701 Domain
Part 1	**Security Foundations**	
Mod 1	Introduction to Information Security	General Concepts (1.0)
Mod 2	Pervasive Attack Surfaces and Controls	Threats, Vulns & Mitigations (2.0)
Part 2	**Cryptography**	
Mod 3	Fundamentals of Cryptography	General Concepts (1.0)
Mod 4	Advanced Cryptography	General Concepts (1.0)
Part 3	**Device Security**	
Mod 5	Endpoint Vulnerabilities, Attacks, and Defenses	Threats, Vulns & Mitigations (2.0)
Mod 6	Mobile and Embedded Device Security	Security Architecture (3.0)
Mod 7	Identity and Access Management (IAM)	Security Architecture (3.0)
Part 4	**Infrastructure and Architectures**	
Mod 8	Infrastructure Threats and Security Monitoring	Security Operations (4.0)
Mod 9	Infrastructure Security	Security Architecture (3.0)
Mod 10	Wireless Network Attacks and Defenses	Threats, Vulns & Mitigations (2.0)
Mod 11*	Cloud and Virtualization Security	Security Architecture (3.0)
Part 5	**Operations and Management**	
Mod 12*	Vulnerability Management	Security Operations (4.0)
Mod 13*	Incident Preparation and Investigation	Security Operations (4.0)
Mod 14*	Oversight and Operations	Sec. Program Mgmt & Oversight (5.0)
Mod 15*	Information Security Management	Sec. Program Mgmt & Oversight (5.0)

** Modules marked with an asterisk are not covered in class lectures but are available for project exploration.*

Step 2: How to Come Up with a Project Idea

Many students feel stuck at this stage. Here is a structured four-step approach:

1. **Start with a scenario, not a technology.** Imagine a fictional organization: a small medical clinic, a law firm, a school district, a retail chain with an online store. What real-world security challenges would they face?
2. **Pick a threat.** What is the most realistic attack against that organization? Ransomware? Phishing leading to credential theft? An insider exfiltrating patient data? A compromised IoT device pivoting into the corporate network?
3. **Design and build the defense.** What combination of technologies, policies, and configurations would protect against that threat? This becomes your project. You must actually build the scenario (the organization's environment), demonstrate how the threat impacts it, implement the defenses, and then show that those defenses are effective.
4. **Map to modules.** Check which course modules your defense touches. You need at least three. If you only have two, consider adding a monitoring, incident response, or governance component.

> **Project Requirement: Build, Attack, Defend**
>
> Your project must include three phases: (1) Build the scenario — replicate the fictional organization's environment using VMs and virtual networks. (2) Demonstrate the threat — show how the chosen attack actually impacts the environment before defenses are in place. (3) Implement and verify defenses — deploy your security controls and demonstrate their effectiveness through testing.

> **Brainstorming Prompt**
>
> Complete this sentence: "A [type of organization] is vulnerable to [specific threat]. We will replicate this environment, demonstrate the attack, and then protect it by implementing [defense 1], [defense 2], and [defense 3]. We will verify effectiveness by [testing method]." That sentence becomes your mission statement.

Step 3: Define Your Mission Statement

Based on your brainstorming, write a one-paragraph mission statement that clearly states: the type of organization, the security problem you are solving, how the threat will be demonstrated, and the key technologies you will use to defend against it. See Appendix A for 10 detailed project examples with module references.

Section 3: Understanding Agile Project Management on GitHub

This section explains the Agile concepts you will use to manage your project. If you have never used GitHub or Agile before, read this section carefully before setting anything up.

The Big Picture: How the Pieces Fit Together

Think of your project as building a house. You would not try to do everything at once. You would plan phases (foundation first, then framing, then electrical, etc.) and within each phase you complete specific tasks. Agile project management works the same way.

Here is how every concept connects:

Concept	What It Is	GitHub Feature	Analogy
Milestone	A major project goal or deliverable phase	Milestone (under Issues tab)	"Finish the foundation"
Sprint (Iteration)	A 2-week time box of work	Iteration (on the Project Board)	"What we build this pay period"
User Story	A goal from a user's perspective (the "why")	Issue labeled "user-story" (parent)	"A room in the house"
Task / Issue	A specific piece of work (the "how")	Sub-Issue under a User Story	"Install a window in that room"
Story Points	Effort/complexity estimate for a User Story	Estimate field (parent issue)	"How hard is that room to build"
Project Board	Visual board showing status of all work	GitHub Projects (board view)	"The whiteboard on the job site"

Milestones and Sprints: They Are Not the Same Thing

This is a common source of confusion, so let us be precise.

What Is a Milestone?

A Milestone represents a major project goal or deliverable phase. It answers the question: "What do we need to have accomplished by this date?" A Milestone has a name, a description, and a due date. In GitHub, you create Milestones under the Issues tab of your repository.

Examples of Milestones:

- "Milestone 1: Project Setup and Planning" — Repository, wiki, architecture diagram, and proposal complete.
- "Milestone 2: Base Infrastructure Deployment" — All VMs installed, network segments configured, basic connectivity verified.
- "Milestone 3: Security Implementation and Attack Demonstration" — Defenses deployed, threat demonstrated, effectiveness tested.

- "Milestone 4: Documentation and Presentation" — Final report, wiki, and presentation complete.

What Is a Sprint (Iteration)?

A Sprint (also called an Iteration on the GitHub Project Board) is a fixed-length work period — in our class, two weeks. During a sprint, your team commits to completing a set of tasks. At the end you review what was done and plan the next sprint.

Sprints are configured as Iterations on the Project Board. Each Iteration has a start date and end date.

How They Relate

A Milestone can span multiple Sprints. For example, "Milestone 2: Base Infrastructure Deployment" might require work across Sprint 1 and Sprint 2 because the infrastructure is complex. Conversely, in a single Sprint you might complete the remaining tasks for one Milestone and begin tasks for the next.

	Sprint 1 (Wk 2–3)	Sprint 2 (Wk 4–5)	Sprint 3 (Wk 6–7)
Milestone 1: Setup	Complete all tasks		
Milestone 2: Infrastructure	Begin VM setup	Finish network config	
Milestone 3: Security & Testing		Begin defenses	Test & document

In GitHub: Two Separate Fields

When you create an Issue in a repository that is linked to a GitHub Project, you will see two separate fields:

- **Milestone:** Set in the Issue sidebar (repository-level). Assign the Issue to the Milestone it contributes to (e.g., "Milestone 2: Base Infrastructure Deployment").
- **Iteration (Sprint):** Set on the Project Board (project-level). Assign the Issue to the Sprint during which you plan to work on it (e.g., "Sprint 1").

> **Key Rule**
> Every Issue must have both a Milestone and an Iteration assigned. The Milestone tells you WHAT goal the work contributes to. The Iteration tells you WHEN you plan to do it. An Issue without both will be difficult to track.

How to Define Good Milestones

Each Milestone should have a clear, measurable goal. Ask: "How will we know this Milestone is done?" If you cannot answer that, the goal is too vague.

- Good: "All VMs are deployed, network segments are configured, and connectivity between zones is verified with screenshots."
- Vague: "Work on infrastructure." (What specifically? How do you know when it is done?)

Why Does GitHub Call Everything an “Issue”?

In GitHub, the word "Issue" is the name of the platform’s built-in tracking feature. It is used for everything: bugs, tasks, feature requests, questions, and user stories. The name comes from software development, where teams originally used Issues to track bugs and problems — but over time the feature expanded to cover all types of work.

In this class, you will use Issues in two ways:

- **User Story Issues (Parent Issues):** These represent high-level goals. Label them "user-story" so you can tell them apart from tasks. Assign story points (Estimate) to these.
- **Task Issues (Sub-Issues):** These represent the specific steps to complete a User Story. Create them as sub-issues under the parent. Assign Size (XS–XL) to these.

The terms "task," "issue," and "sub-issue" are often used interchangeably in conversation. Just remember: in GitHub, the feature is always called an Issue, regardless of what it represents.

User Stories and Tasks: The Full Breakdown

User Stories: The “Why”

A User Story describes a goal from the perspective of someone who benefits from the system. It follows this template:

"As a [role], I want to [action] so that [benefit]."

For a security project, examples include:

- "As a network administrator, I want to segment the network into VLANs so that a compromise in one zone does not spread to others."
- "As a security analyst, I want to deploy a SIEM and configure custom alert rules so that I can detect brute-force login attempts across all monitored systems."
- "As a penetration tester, I want to demonstrate a phishing attack against the test environment so that we can measure whether our email filtering and user training controls are effective."

In GitHub, create a User Story as an Issue, label it "user-story," and assign it an Estimate (story points). This Issue becomes a parent that holds sub-issues (tasks).

Tasks (Sub-Issues): The “How”

Tasks are the specific, actionable steps needed to complete a User Story. In GitHub, create them as Sub-Issues under the parent User Story.

Example: If your User Story is “As a network admin, I want to segment the network into VLANs...”, the tasks might be:

1. Install and configure pfSense VM with WAN and LAN interfaces
2. Create VLAN 10 (Corporate) with IP range 10.10.10.0/24
3. Create VLAN 20 (IoT) with IP range 10.10.20.0/24

4. Create VLAN 30 (Guest) with IP range 10.10.30.0/24
5. Configure inter-VLAN firewall rules (deny IoT → Corporate, allow Corporate → Internet)
6. Test isolation between VLANs, capture screenshots, and document results on the Wiki

The Relationship Visualized

Level	Example	GitHub Feature
Milestone	Milestone 2: Infrastructure	Milestone (with due date)
└ **User Story**	"As an admin, I want VLANs..."	Parent Issue (label: user-story)
└ **Task 1**	Install pfSense VM	Sub-Issue [Sprint 1]
└ **Task 2**	Create VLAN 10	Sub-Issue [Sprint 1]
└ **Task 3**	Configure firewall rules	Sub-Issue [Sprint 2]

Story Points: Where Do They Go?

> **The Rule for This Class**
> Assign story points (the Estimate field) ONLY to User Stories (parent issues). Do NOT assign story points to individual tasks (sub-issues). Instead, use the Size field (XS, S, M, L, XL) on tasks to indicate relative effort.

Why This Approach?

Story points measure the total effort, complexity, and uncertainty of delivering a complete goal (the user story), not the effort of each individual step. Think of it this way: if you are estimating how hard it is to build a bathroom, you give one estimate for the whole bathroom (say, 5 points). You do not score each fixture separately and add them up. The story points capture the overall picture, including risks and unknowns.

How to Assign Points: A Practical Guide

Use the following scale. When deciding between two values, ask: "How much uncertainty is there? Have we done something like this before?"

Pts	Complexity	Criteria	Example User Story
1	Trivial	Exactly one step, no uncertainty, takes under 30 minutes	"Update README with team names and project title"
2	Small	1–2 straightforward steps, you have done this before or a clear tutorial exists	"Install a single VM from a known image and verify network connectivity"
3	Medium	3–4 steps, some configuration research required, low-to-moderate uncertainty	"Configure pfSense with NAT rules and verify LAN→WAN traffic"
5	Large	5+ steps, involves integrating multiple components, moderate uncertainty, may require troubleshooting	"Deploy Wazuh SIEM, install agents on two endpoints, and create a custom alert rule for failed SSH logins"
8	Very Large	Many steps, high uncertainty, dependencies on other stories, team has no prior experience with the technology	"Build a two-tier PKI with offline Root CA, online Sub CA, issue certs to a web server, and verify client trust chain"

When Should You Use 8 Points?

An 8-point story should be rare. It signals that the User Story is at the upper boundary of what your team should attempt in a single story. Before assigning 8 points, ask these questions:

1. Can this story be split into two smaller stories that each deliver independent value? If yes, split it.
2. Does the team have zero prior experience with the core technology? If yes, 8 points may be justified because of the learning curve.

3. Are there external dependencies or unknowns that could block progress? If yes, 8 points is appropriate to account for that risk.

> **Warning**
> If you find yourself assigning 8 points to most of your User Stories, your stories are too large. Break them down. A healthy sprint should have a mix of 2-, 3-, and 5-point stories, with at most one 8-point story.

What About Tasks?

For tasks (sub-issues), use the Size field on the Project Board instead of points:

- XS = Less than 30 minutes of work
- S = About 1 hour
- M = A few hours
- L = A full work session (half a day)
- XL = Too big. Break it into smaller tasks.

Putting It All Together: A Worked Example

Here is a concrete example for a team building a "Secure Small Business Network" project.

Sprint 1 Planning

Milestone: "Milestone 2: Base Infrastructure Deployment"

Sprint (Iteration): Sprint 1

User Story A (5 points): "As a network administrator, I want to deploy a segmented network with a DMZ so that the web server is isolated from internal systems."

Tasks (Sub-Issues):

1. Install pfSense VM and configure WAN/LAN interfaces [Size: M, Assignee: Alice]
2. Create DMZ network segment on pfSense [Size: S, Assignee: Alice]
3. Deploy Ubuntu Server VM in DMZ and install Apache [Size: M, Assignee: Bob]
4. Deploy Windows 10 VM on the LAN segment [Size: S, Assignee: Carlos]
5. Write initial firewall rules: LAN → Internet (allow), DMZ → LAN (deny) [Size: M, Assignee: Alice]
6. Test and document: verify web server reachable from WAN, verify DMZ cannot reach LAN [Size: M, Assignee: Bob + Carlos]

User Story B (2 points): "As a team member, I want our project wiki and repository to be organized so that we can document our progress efficiently."

Tasks (Sub-Issues):

1. Create repository directory structure per Appendix D [Size: S, Assignee: Carlos]
2. Set up Wiki home page with table of contents [Size: S, Assignee: Carlos]
3. Write project mission statement in README.md [Size: XS, Assignee: Alice]
4. Create Wiki page for Sprint 1 progress notes [Size: XS, Assignee: Bob]

Total sprint commitment: 7 story points. Note how every team member is assigned technical tasks, and the documentation tasks are spread across the team.

Section 4: Setting Up Your GitHub Environment

Your GitHub repository is your team's central hub. Follow these steps in order.

Step 1: Create Your Repository

1. Log in to GitHub.com.
2. Click the + icon (top-right) and select New repository.
3. **Repository name:** Use the naming convention **cis3353_s26_XXX_XXX_XX** where XXX or XX are 2–3 letter initials of each team member. For example, a team with Alice Brown, Carlos Diaz, and Eve Ng would use: cis3353_s26_AB_CD_EN.
4. Description: Briefly describe your project objective.
5. Select Private. This is mandatory.
6. Check the box for Add a README file.
7. Click Create repository.
8. Add Collaborators: Go to Settings > Collaborators and teams. Add your teammates and the instructor accounts: gdparra-edu and cyberknowledge.

Step 2: Enable the Wiki

1. In your repository, click the Settings tab.
2. In the "Features" section, check the box next to Wikis.
3. Click the new Wiki tab and create the first page.
4. Set up your Wiki home page as a table of contents linking to sub-pages (Sprint Notes, Architecture Decisions, Final Report, etc.).

Step 3: Create a Project Board

1. Go to the Projects tab in your repository.
2. Click New project and choose the "Iterative development" template (or "Board" if unavailable).
3. This gives you columns: Backlog, Ready, In Progress, In Review, Done.
4. Configure Iterations (Sprints) with two-week durations matching your class schedule.

Step 4: Create Your Milestones

Before creating any Issues, set up your Milestones:

1. Go to Issues > Milestones > New milestone.
2. Create one Milestone per major project phase (see examples in Section 3).
3. Set the due date (use the dates from Canvas) and describe the deliverables for that Milestone.
4. Repeat for all planned Milestones.

Step 5: Create Issues (User Stories and Tasks)

Now populate your first sprint:

1. Create a User Story Issue: Title it clearly, label it “user-story,” assign it to a Milestone, and set the Estimate (story points) on the Project Board.
2. Create Task Sub-Issues: Open the User Story, click “Add sub-issue,” and create each task. Assign each task to a team member, set a Size (XS–XL), and assign it to the same Milestone.
3. Add all Issues to the Project Board and set the Iteration (Sprint) field.

Step 6: Establish Your Directory Structure

Create your repository’s directory structure from day one. See Appendix D for the recommended layout.

Section 5: Project Proposal Checklist

Due date: See Canvas. Your proposal should cover the following:

- ☐ **Project Title & Mission Statement**
- ☐ **Course Modules Integrated:** List the 3+ modules and explain how each connects to the project.
- ☐ **System Architecture Diagram:** VMs, networks, firewalls, key services.
- ☐ **Threat Demonstration Plan:** How you will demonstrate the attack/threat before defenses are applied.
- ☐ **Defense and Verification Plan:** What controls you will implement and how you will prove they work.
- ☐ **Resources Required:** Specific VMs, Docker images, or cloud services.
- ☐ **Key Deliverables:** Functional environment, final report (Wiki), organized repo, final presentation.
- ☐ **Milestone and Sprint Plan:** All Milestones defined. At least Sprint 1 fully planned with user stories and tasks.
- ☐ **Questions for the Instructor:** Document on a Wiki page.

Appendix A: Sample Project Ideas (8th Edition)

Below are ten project ideas updated for the 8th Edition module numbers. Each integrates at least three modules and requires you to build the scenario, demonstrate the threat, and implement/verify defenses.

1. Secure Small Business Network with DMZ and IDS

Modules: 2 (Attack Surfaces), 8 (Infrastructure Threats & Monitoring), 9 (Infrastructure Security), 10 (Wireless)

Scenario: A small accounting firm with 15 employees needs a public-facing client portal, an internal network for staff, and secure wireless access. The firm handles sensitive financial data and must comply with basic security standards.

Threat Demonstration: From an external attacker VM, perform a port scan and attempt to access the internal network from the DMZ. Show that without proper segmentation and IDS, an attacker who compromises the web server can pivot into the LAN.

Defense & Verification: Configure pfSense with three zones (WAN, DMZ, LAN), deploy Snort/Suricata IDS on the DMZ interface, and set up WPA3-Enterprise for wireless. Verify that IDS alerts fire on the port scan, that DMZ-to-LAN traffic is blocked, and that only authenticated wireless clients can connect.

Resources: 1x pfSense, 1x Ubuntu Server (web portal), 1x Windows 10 (internal client), 1x Kali Linux (attacker). VirtualBox/VMware.

2. Active Directory Hardening with Attack Simulation

Modules: 5 (Endpoint Vulnerabilities), 7 (IAM), 8 (Infrastructure Threats & Monitoring)

Scenario: A mid-size company with 200 employees uses Active Directory. Recent news of credential-stuffing attacks has prompted management to request a security review of the AD environment.

Threat Demonstration: Using Kali Linux, perform a password spray attack against the domain. Demonstrate Kerberoasting to extract service account ticket hashes. Show lateral movement from a compromised workstation to the Domain Controller using tools like Mimikatz (in a controlled lab environment).

Defense & Verification: Apply GPOs to enforce strong passwords, account lockout, LAPS (Local Admin Password Solution), and disable NTLM where possible. Configure Windows Event Forwarding to a central log collector. Verify that the same attacks now fail or trigger alerts.

Resources: 1x Windows Server 2022 (DC), 2x Windows 10/11 (domain clients), 1x Kali Linux (attacker), 1x Ubuntu Server (log collector).

3. PKI with Mutual TLS Authentication

Modules: 3 (Cryptography Fundamentals), 4 (Advanced Cryptography), 9 (Infrastructure Security)

Scenario: A healthcare organization needs to secure communications between its patient portal and internal database server, and ensure that only authorized devices can connect to the internal API.

Threat Demonstration: Show that without PKI, traffic between the portal and database is unencrypted (using Wireshark to capture credentials in transit). Demonstrate a man-in-the-middle attack using a self-signed certificate.

Defense & Verification: Build a two-tier PKI (offline Root CA, online Subordinate CA). Issue server certificates for TLS on the web server and client certificates for mutual TLS on the API. Verify that Wireshark shows encrypted traffic, that the MITM attack fails with certificate pinning, and that clients without valid certificates are rejected.

Resources: 1x Windows Server (offline Root CA), 1x Windows Server (Sub CA + IIS), 1x Windows 10 (client), 1x Kali Linux (attacker for MITM).

4. SIEM Deployment with Real-Time Attack Detection

Modules: 2 (Attack Surfaces), 5 (Endpoint Vulnerabilities), 8 (Infrastructure Threats & Monitoring)

Scenario: A financial services startup needs centralized security monitoring. They have Windows workstations, a Linux web server, and a pfSense firewall, and need to detect brute-force attacks, privilege escalation, and data exfiltration attempts.

Threat Demonstration: From a Kali VM, perform SSH brute-force attacks against the Linux server, attempt privilege escalation using a known exploit, and simulate data exfiltration via DNS tunneling.

Defense & Verification: Deploy Wazuh SIEM with agents on all endpoints. Create custom detection rules for: failed login threshold alerts, sudo privilege escalation events, and unusual DNS query volumes. Build dashboards showing attack timelines. Verify each attack generates the expected alert within 60 seconds.

Resources: 1x Ubuntu Server (Wazuh manager), 1x Ubuntu Server (web server), 1x Windows 10 (endpoint), 1x pfSense, 1x Kali Linux.

5. Web Application Security: Attack and Defense

Modules: 5 (Endpoint Vulnerabilities), 8 (Infrastructure Threats), 9 (Infrastructure Security)

Scenario: An e-commerce company is deploying a new web application. Before going live, the security team must identify vulnerabilities, deploy defenses, and prove the application is secure against the OWASP Top 10.

Threat Demonstration: Deploy OWASP Juice Shop or DVWA. Successfully exploit at least three vulnerability categories: SQL Injection, Cross-Site Scripting (XSS), and Broken Authentication. Document each exploit with screenshots and the specific payloads used.

Defense & Verification: Deploy ModSecurity WAF with OWASP Core Rule Set. Configure input validation rules. Implement rate limiting for login attempts. Re-run the same exploits and demonstrate that the WAF blocks each attack. Include WAF logs as evidence.

Resources: Docker containers for Juice Shop/DVWA and ModSecurity, or 1x Ubuntu Server hosting both. 1x Kali Linux for testing.

6. Zero-Trust Network Segmentation

Modules: 7 (IAM), 8 (Infrastructure Threats), 9 (Infrastructure Security), 10 (Wireless)

Scenario: A smart building management company has corporate workstations, IoT sensors (HVAC, cameras), and a guest wireless network all on the same flat network. After a competitor was breached via a compromised IoT camera, they need to implement zero-trust segmentation.

Threat Demonstration: On the flat network, show that an IoT device (simulated) can communicate freely with corporate workstations. Demonstrate ARP spoofing from the IoT VLAN to intercept corporate traffic.

Defense & Verification: Create four VLANs (Corporate, IoT, Guest, Management) on pfSense. Implement inter-VLAN firewall rules following least-privilege. Add 802.1X port-based authentication for the corporate VLAN. Verify that IoT devices cannot reach corporate hosts, that guest devices can only reach the internet, and that ARP spoofing across VLANs fails.

Resources: 1x pfSense, 4+ lightweight VMs (TinyCore Linux, or containers) representing devices in each VLAN, 1x Kali Linux (attacker).

7. Secure Remote Access with VPN and MFA

Modules: 7 (IAM), 9 (Infrastructure Security), 11 (Cloud & Virtualization Security)

Scenario: A consulting firm with 50 remote employees needs secure access to internal file shares and an intranet application. Management requires multi-factor authentication for all remote connections.

Threat Demonstration: Show that without VPN, internal resources are inaccessible from outside. Then demonstrate a credential-theft scenario: even with stolen VPN credentials, an attacker without the second factor cannot connect.

Defense & Verification: Configure OpenVPN on pfSense with certificate-based authentication plus TOTP (using Google Authenticator or similar). Set up split tunneling so only internal traffic routes through the VPN. Verify: remote client accesses internal resources through VPN; stolen credentials without TOTP are rejected; split tunneling correctly routes only internal traffic.

Resources: 1x pfSense (VPN server), 1x Ubuntu Server (intranet + file share), 1x Windows 10 (remote client), 1x Windows 10 (attacker).

8. Mobile Device Security and BYOD Policy Enforcement

Modules: 6 (Mobile & Embedded Device Security), 7 (IAM), 14 (Oversight & Operations)

Scenario: A university department allows faculty to use personal devices to access email and shared drives. After a faculty member's phone was lost with unencrypted university data, the department needs an MDM solution and a formal BYOD policy.

Threat Demonstration: Show that an unmanaged device can access university resources without any security controls. Demonstrate what happens when a device is "lost": data remains accessible, no remote wipe capability exists.

Defense & Verification: Deploy an open-source MDM (e.g., FleetDM or Headwind MDM). Enroll a test device. Enforce policies: passcode requirement, storage encryption, app blacklist, and remote wipe capability. Write a formal BYOD policy document. Verify each policy is enforced and demonstrate a successful remote wipe.

Resources: 1x Ubuntu Server (MDM server), 1x Android device or Android Studio emulator, documentation for BYOD policy.

9. Vulnerability Assessment, Exploitation, and Remediation

Modules: 2 (Attack Surfaces), 5 (Endpoint Vulnerabilities), 12 (Vulnerability Management)

Scenario: A company's IT audit revealed that their server infrastructure has not been patched in over a year. The security team must assess the current risk, demonstrate the impact of key vulnerabilities, apply remediation, and verify the fixes.

Threat Demonstration: Scan the vulnerable target with OpenVAS or Nessus Essentials. Select at least three Critical/High vulnerabilities and successfully exploit them using Metasploit. Document the full attack chain for each exploit.

Defense & Verification: For each exploited vulnerability, apply the appropriate patch or configuration hardening. Document the remediation rationale (why this fix, what it changes). Re-run the vulnerability scan and re-attempt the exploits to confirm they are mitigated. Produce a before/after comparison report.

Resources: 1x Kali Linux (scanner + attacker), 1x Metasploitable 2 or VulnHub machine (target), 1x Ubuntu Server (patched target for comparison).

10. Honeypot Deployment with Threat Intelligence Analysis

Modules: 2 (Attack Surfaces), 8 (Infrastructure Threats & Monitoring), 13 (Incident Preparation)

Scenario: A cybersecurity research team at a university wants to understand the current threat landscape by deploying honeypots that mimic common services (SSH, HTTP, SMB) and analyzing the attacks they attract.

Threat Demonstration: Deploy the honeypot with default/weak configurations and expose it to the internet. After 48–72 hours of data collection, analyze the attack patterns: credential dictionaries used, exploit attempts, geographic origins of attackers, and automated vs. targeted attack behavior.

Defense & Verification: Using the intelligence gathered, write detection rules (Snort/Suricata or Wazuh) that would catch the top 5 most common attack patterns observed. Deploy these rules on a production-like environment and verify they trigger correctly when the same attack patterns are replayed from a test machine.

Resources: 1x Cloud VM (AWS/GCP/DigitalOcean) with public IP for the honeypot. Docker containers (T-Pot or Cowrie). 1x Ubuntu Server with Wazuh/Suricata for rule testing. 1x Kali Linux for replaying attacks.

Appendix B: GitHub First-Timer's Quickstart

If you have never used GitHub before, here is the minimum you need to know.

Key Terminology

- **Repository (Repo):** A project folder on GitHub containing your files, history, and settings.
- **Issue:** GitHub's built-in tracking item. Used for user stories, tasks, bugs, and questions. Think of it as a task card.
- **Sub-Issue:** An Issue nested under a parent Issue. Used to break User Stories into Tasks.
- **Milestone:** A goal container that groups Issues by deliverable phase, with a due date.
- **Iteration (Sprint):** A time-boxed period on the Project Board. Configured in the Project settings.
- **Project Board:** A Kanban-style board (Backlog → In Progress → Done) that visualizes your Issues.
- **Wiki:** Built-in documentation pages attached to your repository. Your final report lives here.
- **Label:** A color-coded tag for categorizing Issues (e.g., "user-story," "bug," "documentation").

The Workflow in 6 Steps

1. Create Milestones (one per major project phase) with due dates and deliverable descriptions.
2. Configure Iterations (Sprints) on the Project Board with two-week durations.
3. Create User Story Issues (labeled "user-story"), assign Estimate (points) and a Milestone.
4. Create Task Sub-Issues under each User Story, assign team members, Size (XS–XL), and the same Milestone.
5. Set the Iteration (Sprint) field for each Issue on the Project Board.
6. Move cards across the board as work progresses: Backlog → Ready → In Progress → In Review → Done.

Appendix C: Frequently Asked Questions

Q: Do I assign story points to tasks or to user stories?

Assign story points (Estimate) only to User Stories (parent issues). Tasks get a Size (XS–XL) instead. Story points measure the overall effort of delivering a complete goal, not individual steps.

Q: What if a task turns out to be bigger than expected?

If a task is too large, break it into smaller sub-issues. If a User Story is too large (8+ points), consider splitting it into two separate User Stories.

Q: What is the difference between a Milestone and an Iteration (Sprint)?

A Milestone is a project goal with a due date (e.g., "Infrastructure Deployed"). An Iteration/Sprint is a two-week time box during which you work on tasks. A Milestone can span multiple Sprints. When you create an Issue, you assign it to a Milestone (what goal it contributes to) and an Iteration (when you plan to do it).

Q: How many story points should we plan per sprint?

For a 2–3 person team in a two-week sprint, plan 8–15 points total. Start conservatively in Sprint 1 (around 7–10 points). As you learn your team's pace, adjust.

Q: Can I change the sprint plan mid-sprint?

Yes. If you finish early, pull tasks from the Backlog. If something takes longer than expected, move incomplete items to the next Sprint's Iteration. Document the change in your Wiki sprint notes.

Q: Why are tasks, issues, and sub-issues all called different things?

In GitHub, everything is an "Issue" — that is the name of the platform feature. "Task" is the Agile term for a specific piece of work. "Sub-issue" describes the parent-child relationship. They refer to the same thing viewed from different angles: GitHub calls it an Issue, Agile calls it a Task, and when it is nested under another Issue we call it a Sub-Issue.

Q: Does every team member need to do technical work?

Yes. Every team member must contribute to building, configuring, and testing the systems. Roles like "Project Manager" or "Documentation Lead" define who coordinates those areas, but every member must also be hands-on with the technical implementation.

Q: How do I pick a project idea?

Consider three factors: your team's interest (you will spend weeks on this), the resources available (VMs, cloud credits), and the module coverage (minimum three). Use the brainstorming framework in Section 2 to generate custom ideas if the examples in Appendix A do not fit.

Appendix D: Recommended Repository Structure

Establish this structure from day one. A well-organized repository makes documentation and grading easier.

```
/
├── configs/
│   ├── firewall/
│   ├── server/
│   └── network/
├── docs/
│   ├── architecture-diagram.png
│   └── sprint-notes/
├── scripts/
│   └── setup/
├── evidence/
│   └── README.md  (hashes and download links only)
├── reports/
│   └── final_report_draft.md
└── README.md
```

- **/configs/:** Firewall rules exports, server configuration files, network diagrams.
- **/docs/:** Architecture diagrams, sprint retrospective notes, design decisions.
- **/scripts/:** Any automation or setup scripts your team writes.
- **/evidence/:** DO NOT upload raw evidence files. Only store hashes and download links here.
- **/reports/:** Drafts of your final report (the final version lives on the Wiki).
- **README.md:** High-level project summary, team members, and a link to the Wiki.

Appendix E: Examples of High-Quality Documentation

Exemplary README.md Files

- TensorFlow — A clear, professional "front door" to a complex project.
- Awesome Lists — Simple markdown used for excellent organization.

Exemplary GitHub Wikis

- Hystrix — Latency and Fault Tolerance for Distributed Systems.
- Visual Studio Code — Comprehensive Wiki with excellent navigation.
- Homebrew — Clean, concise Wiki focused on solving specific problems.

Study these examples for inspiration on how to structure your own README and Wiki pages.